Relatório Técnico e Estratégico

Aplicação mobile de chat com IA em modelo BYOK

Versão: 1.1
Data: 1 de setembro de 2026
Arquitetura selecionada: Opção A — comunicação direta entre a aplicação e os providers de IA
Plataformas: iOS 15+ e Android 10+
Tecnologia principal: React Native + Expo + TypeScript
Classificação: Documento de arquitetura, segurança, produto e implementação
Estado: Recomendação para validação técnica antes do desenvolvimento do MVP

> Este documento transforma a proposta inicial num plano de decisão. Não constitui garantia de segurança absoluta, compatibilidade permanente com APIs de terceiros ou estimativa contratual de custos.

1. Decisão executiva

Recomenda-se avançar com a **Opção A** para o MVP: a aplicação comunica diretamente com o provider escolhido e o backend próprio, caso exista, fica limitado a configuração não sensível.

Esta opção é a mais coerente com uma proposta de valor baseada em BYOK e privacidade perante o operador da aplicação. Evita que as mensagens, respostas, áudio e API keys tenham de passar por uma infraestrutura própria. Em contrapartida, transfere para a aplicação mobile responsabilidades normalmente assumidas por um backend:

- gestão de credenciais no dispositivo;
- compatibilidade com APIs e formatos de streaming;
- normalização de erros e utilização;
- controlo de retries e pedidos duplicados;
- exposição de tráfego e de uma API key num dispositivo que pode estar comprometido;
- ausência de controlo central sobre abuso, revogação e limites de utilização.

A decisão é tecnicamente viável, desde que a comunicação comercial seja precisa:

> A aplicação não funciona como proxy das conversas. O conteúdo enviado pelo utilizador é transmitido diretamente ao provider selecionado com a API key configurada no dispositivo. O operador da aplicação não deve receber esse conteúdo no MVP.

1.1. Recomendação de lançamento

O MVP deve começar com **um único provider**, streaming, histórico local, Secure Store e TTS manual. STT deve avançar apenas depois de ser validado num development build e em dispositivos físicos.

O lançamento não deve depender do backend auxiliar. Catálogo, preços e configurações essenciais devem ter fallback local. O backend pode ser introduzido para distribuição de configuração, mas não deve tornar-se um ponto obrigatório no caminho de uma conversa.

1.3. Provider inicial aprovado

O provider inicial será **OpenAI**, através da **Responses API**, com o modelo configurado inicialmente como `gpt-5.6-luna`. O identificador deve permanecer numa configuração de catálogo, e não disperso pela UI, para permitir atualização caso o alias, preço ou disponibilidade mude.

De acordo com a documentação consultada em 2 de setembro de 2026, `gpt-5.6-luna` é apresentado pela OpenAI como modelo para workloads sensíveis a custo e volume, com janela de contexto de 1,05 milhões de tokens e preço publicado de referência de $0,20 por milhão de tokens de entrada e $1,20 por milhão de tokens de saída. Estes valores devem ser tratados como configuração versionada, não como promessa comercial.

O adapter inicial deve usar `responses.create`, streaming e `store: false` por defeito. A aplicação continuará a guardar o histórico localmente; não deve usar a Conversations API da OpenAI para representar os projetos locais sem uma decisão específica sobre retenção de dados no provider.

1.2. Critérios para continuar

A fase de validação técnica só deve ser considerada concluída quando for possível, num iPhone e num Android físicos:

1. guardar e remover uma key sem a escrever em SQLite, AsyncStorage, logs ou estado persistido;
2. enviar uma mensagem diretamente para o provider;
3. receber e renderizar streaming;
4. cancelar uma geração sem deixar a UI num estado inconsistente;
5. interromper e retomar uma operação de voz de acordo com as permissões da plataforma;
6. reiniciar a aplicação e recuperar o histórico local;
7. confirmar os fluxos de erro de autenticação, quota, rede e modelo inválido;
8. demonstrar que nenhum payload de conversa chega ao backend próprio.

2. Objetivos, não objetivos e pressupostos

2.1. Objetivos do produto

O produto deve ser:

- simples para quem já possui uma API key;
- claro para quem não conhece BYOK;
- visualmente distinto de clientes genéricos;
- privado perante o operador da aplicação;
- multiplataforma;
- rápido na resposta percebida;
- extensível a vários providers;
- preparado para voz sem enviar áudio de forma silenciosa;
- transparente quanto a utilização, preços e limitações.

2.2. Objetivos do MVP

O MVP deverá permitir:

- configurar uma ou mais API keys;
- validar uma configuração com uma chamada controlada ao provider;
- escolher provider e modelo;
- criar, renomear e apagar conversas;
- criar, renomear, arquivar e apagar projetos que agregam conversas do mesmo tema;
- mover uma conversa para um projeto ou mantê-la fora de projetos;
- iniciar uma nova conversa dentro de um projeto usando como contexto todas as conversas desse projeto;
- enviar mensagens e receber streaming;
- cancelar, regenerar, editar e reenviar mensagens;
- copiar e partilhar conteúdo por ação explícita do utilizador;
- manter o histórico local;
- ditar a prompt através de um microfone no composer, rever a transcrição e só depois enviar;
- ouvir uma resposta através de TTS manual;
- escolher nas Definições a voz da IA, o idioma e a velocidade disponíveis para a IA;
- apagar todos os dados locais;
- usar a aplicação sem conta;
- consultar uma estimativa de tokens e custo quando o provider fornecer usage e existir preço conhecido.

2.3. Fora do MVP

Ficam fora da primeira versão, salvo decisão posterior:

- sincronização entre dispositivos;
- conta de utilizador;
- proxy próprio de mensagens;
- colaboração em conversas;
- marketplace ou runtime de plugins;
- execução de agentes ou ferramentas com acesso externo;
- anexos e visão;
- hands-free contínuo;
- autoplay de áudio;
- recuperação de histórico após desinstalação;
- administração central de API keys;
- garantias de funcionamento offline para gerar respostas.

2.4. Pressupostos que devem ser confirmados

- o utilizador aceita obter e pagar diretamente as suas keys junto dos providers;
- o provider escolhido permite uso compatível com uma aplicação mobile e com os termos aplicáveis;
- o sistema operativo fornece armazenamento seguro suficiente para o nível de ameaça definido;
- as bibliotecas de STT disponíveis suportam iOS 15+ e Android 10+ nos dispositivos-alvo;
- a publicação nas lojas aceita o fluxo de entrada de keys e a explicação de privacidade;
- todos os providers-alvo oferecem um mecanismo de streaming estável e documentado.

3. Arquitetura selecionada

3.1. Visão lógica



3.2. Fluxo de dados aprovado

Dado	Origem	Destino	Justificação	Retenção própria
API key	Utilizador	Secure Store do dispositivo	Autenticar no provider	Até remoção pelo utilizador ou limpeza da app
Prompt	Campo de chat	Provider escolhido	Gerar resposta	Não armazenado pelo backend próprio
Resposta	Provider	Aplicação	Apresentar e guardar localmente	SQLite local até eliminação
Histórico	Aplicação	SQLite local	Experiência e continuidade	Local, até eliminação
Áudio STT	Microfone	Biblioteca/sistema de reconhecimento	Converter voz em texto	Depende da plataforma; deve ser documentado
Texto TTS	Aplicação	Motor de voz do dispositivo ou serviço da plataforma	Ler resposta	Não enviado para analytics
Modelo e preços	Backend/fallback local	Aplicação	Configuração e estimativa	Cache local sem segredos
Telemetria técnica	Aplicação	Serviço escolhido, se adotado	Diagnóstico mínimo	Sem conteúdo e sem credenciais

A tabela deve ser refletida na política de privacidade e atualizada sempre que uma dependência alterar o fluxo de dados.

3.3. Backend próprio

O backend opcional não deve estar no caminho de uma conversa. Endpoints possíveis:

```
GET /health
GET /v1/config
GET /v1/providers
GET /v1/models
GET /v1/pricing
```

Requisitos mínimos:

- HTTPS;
- CORS restritivo para clientes web, sem assumir que CORS protege a app nativa;
- schemas versionados;
- cache e fallback local;
- rate limiting para proteger a infraestrutura;
- autenticação administrativa fora da app pública;
- logs sem query strings sensíveis, bodies ou headers de autorização;
- monitorização de disponibilidade, latência e códigos HTTP;
- política de retenção documentada.

O backend pode observar IP, user-agent, horário e padrões de acesso, mesmo sem observar conversas. Essa realidade deve ser incluída na documentação de privacidade.

4. Arquitetura de software

4.1. Organização recomendada

```
src/
├── app/
│   ├── navigation/
│   ├── providers/
│   └── theme/
├── components/
│   ├── Button/
│   ├── IconButton/
│   ├── MarkdownMessage/
│   ├── EmptyState/
│   └── ErrorState/
├── features/
│   ├── chat/
│   │   ├── components/
│   │   ├── chat.store.ts
│   │   ├── chat.service.ts
│   │   └── chat.types.ts
│   ├── conversations/
│   ├── settings/
│   ├── speech/
│   │   ├── stt.service.ts
│   │   └── tts.service.ts
│   └── onboarding/
├── infrastructure/
│   ├── database/
│   ├── secure-storage/
│   ├── networking/
│   └── logging/
├── providers/
│   ├── types.ts
│   ├── registry.ts
│   ├── anthropic/
│   ├── openai/
│   └── gemini/
├── config/
├── hooks/
├── utils/
└── types/
```

A UI não deve construir URLs, headers, bodies ou parsers de um provider. Esse conhecimento pertence ao respetivo adapter.

4.2. Contrato de provider

```
interface AIProviderAdapter {
  readonly id: ProviderId;
  getModels(options?: RequestOptions): Promise<ModelDefinition[]>;
  validateApiKey(options: RequestOptions): Promise<ValidationResult>;
  sendMessage(request: ChatRequest): Promise<ChatResponse>;
  streamMessage(
    request: ChatRequest,
    handlers: StreamHandlers,
  ): Promise<void>;
}
```

O contrato deve receber a key através de um limite de infraestrutura, e não através de componentes React. Uma forma aceitável é `RequestOptions` conter uma função ou contexto de credencial de curta duração. A key completa não deve ser guardada no Zustand, em props ou em objetos de request que possam ser persistidos.

O adapter deve ser responsável por:

- endpoint e versão da API;
- autenticação;
- serialização de mensagens;
- parsing de streaming;
- normalização de usage;
- tradução de erros;
- limites e capacidades do modelo;
- eventual diferença entre APIs de completions, responses ou contents.

4.3. Eventos internos de streaming

```
type StreamEvent =
  | { type: 'text_delta'; text: string }
  | { type: 'usage'; inputTokens?: number; outputTokens?: number }
  | { type: 'done' }
  | { type: 'error'; error: NormalizedProviderError };
```

A implementação deve lidar com:

- resposta SSE fragmentada entre chunks de rede;
- linhas vazias e eventos keep-alive;
- JSON dividido em mais do que um chunk;
- fim normal sem usage;
- cancelamento pelo utilizador;
- stream interrompido ao entrar em background;
- evento de erro antes de `done`;
- resposta sem texto, mas com tool call ou modalidade não suportada.

O parsing deve ser testado com fixtures capturadas e sanitizadas. Não devem ser incluídas keys, prompts reais ou respostas privadas nos fixtures.

5. Persistência e ciclo de vida dos dados

5.1. SQLite

```
CREATE TABLE projects (
  id TEXT PRIMARY KEY NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  color_token TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  updated_at INTEGER NOT NULL,
  archived INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE conversations (
  id TEXT PRIMARY KEY NOT NULL,
  title TEXT NOT NULL,
  project_id TEXT,
  provider_id TEXT NOT NULL,
  model_id TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  updated_at INTEGER NOT NULL,
  archived INTEGER NOT NULL DEFAULT 0,
  FOREIGN KEY (project_id)
    REFERENCES projects(id)
    ON DELETE SET NULL
);

CREATE TABLE messages (
  id TEXT PRIMARY KEY NOT NULL,
  conversation_id TEXT NOT NULL,
  role TEXT NOT NULL,
  content TEXT NOT NULL,
  status TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  input_tokens INTEGER,
  output_tokens INTEGER,
  estimated_cost REAL,
  FOREIGN KEY (conversation_id)
    REFERENCES conversations(id)
    ON DELETE CASCADE
);
```

Índices esperados:

```
CREATE INDEX messages_conversation_created_idx
  ON messages (conversation_id, created_at);

CREATE INDEX conversations_updated_idx
  ON conversations (updated_at DESC);

CREATE INDEX conversations_project_updated_idx
  ON conversations (project_id, updated_at DESC);
```

Um projeto é um agregado local de conversas sobre o mesmo tema. No MVP, o projeto não é uma conta nem uma entidade sincronizada: contém nome, descrição opcional, cor da paleta, estado e referências para conversas. A eliminação de um projeto deve deixar as conversas intactas, salvo se o utilizador escolher explicitamente eliminá-las também.

O `project_id` não é apenas uma etiqueta de organização: define o contexto funcional de uma nova conversa. Quando o utilizador inicia uma conversa dentro de um projeto, o pedido ao provider deve ser composto por **todas as mensagens de todas as conversas não eliminadas desse projeto, mais o conteúdo novo da conversa atual**. Conversas fora do projeto nunca entram automaticamente nesse contexto.

Requisitos:

- foreign keys ativas;
- migrações numeradas e testadas;
- transação ao criar conversa e primeira mensagem;
- limite ou estratégia para conversas muito grandes;
- paginação local quando necessário;
- eliminação em cascata verificada;
- exportação explícita e controlada pelo utilizador;
- nenhuma API key na base de dados.

SQLite local não equivale automaticamente a uma base de dados cifrada. Para o beta, a cifragem adicional do histórico não é um bloqueador: usar a proteção do dispositivo, manter as API keys no Secure Store, não guardar segredos na base de dados e documentar o risco de extração do sandbox. Depois do beta, deve ser reavaliada uma opção de cifragem compatível com Expo, com impacto documentado em builds, performance, migrações e suporte multiplataforma.

5.2. Estados de mensagem

```
pending
sending
streaming
completed
cancelled
failed
```

A mensagem parcial deve ser persistida quando:

- o stream termina;
- o utilizador cancela;
- ocorre uma falha recuperável;
- a aplicação passa para background e a estratégia definida exige preservar o estado.

Uma transição para `completed` só deve ocorrer depois de o adapter confirmar o fim normal. Cancelar o `AbortController` não deve ser apresentado como erro de provider.

5.3. Eliminação de dados

A ação “Apagar todos os dados” deve:

1. pedir confirmação inequívoca;
2. cancelar streams ativos;
3. parar TTS e STT;
4. apagar tabelas e ficheiros locais da app;
5. remover todas as keys do Secure Store;
6. limpar preferências não sensíveis;
7. invalidar caches locais;
8. apresentar o resultado e eventuais limitações do sistema operativo.

A desinstalação da app, backups do sistema e exportações realizadas pelo utilizador podem seguir regras da plataforma. A documentação não deve prometer eliminação de cópias fora do controlo da aplicação.

6. Gestão de API keys

6.1. Armazenamento

Usar `expo-secure-store`, com um identificador por provider:

```
secure_key_provider_anthropic
secure_key_provider_openai
secure_key_provider_gemini
```

Nunca guardar keys em:

- AsyncStorage;
- SQLite;
- ficheiros JSON;
- estado persistido do Zustand;
- Redux Persist;
- variáveis de ambiente incluídas no bundle;
- logs, analytics, crash reports ou breadcrumbs;
- URLs ou query strings;
- clipboard sem ação explícita do utilizador.

`Secure Store` reduz o risco, mas não protege absolutamente um dispositivo rooted, jailbroken, instrumentado, com malware ou com debugging privilegiado. O texto legal e a UI devem dizer isso sem alarmismo.

6.2. Fluxo de configuração

1. O utilizador escolhe o provider.
2. A app apresenta instruções para obter uma key no site oficial.
3. O utilizador introduz a key num campo protegido.
4. A app valida apenas formato e comprimento plausíveis, sem tentar inferir validade real.
5. A key é gravada no Secure Store.
6. A app executa, sob ação explícita, uma validação direta no provider.
7. A UI mostra apenas estado configurado e, opcionalmente, uma identificação parcial que não seja desnecessária.
8. A key completa deixa de ser exibida.
9. Ao remover, a app apaga a entrada do Secure Store e invalida caches de runtime.

A validação deve evitar uma geração dispendiosa. Quando o provider disponibilizar um endpoint de modelos ou uma operação de baixo custo, essa operação deve ser usada. O resultado “key aceite” não deve ser confundido com “saldo disponível”, “permissão para todos os modelos” ou “conta faturável”.

6.3. Uso em memória

Durante uma chamada, a key terá de existir em memória. A mitigação possível é:

- ler a key apenas no início da operação;
- limitar o seu âmbito a uma função de request;
- não a colocar em logs nem em mensagens de erro;
- não a reter após conclusão ou cancelamento, tanto quanto o runtime permitir;
- não incluir a key em objetos de telemetria;
- evitar dumps de request em ferramentas de debugging de produção.

Não é tecnicamente realista prometer que a key nunca será observável num dispositivo comprometido.

6.4. Headers e URLs

As keys devem ser enviadas em headers de autenticação suportados pelo provider. Não devem ser colocadas em URLs, query strings, deep links ou identificadores de crash. O adapter de cada provider deve ter testes que falhem se um segredo for serializado numa URL.

7. Rede, retries e erros

7.1. Requisitos da camada HTTP

A camada de rede deve suportar:

- HTTPS obrigatório;
- timeout separado de conexão e leitura, quando disponível;
- `AbortController`;
- streaming;
- cancelamento;
- validação de content type;
- limite de tamanho para respostas e erros;
- parsing por provider;
- normalização de usage;
- prevenção de pedidos duplicados;
- logs estruturados e redigidos.

Pinning de certificados não deve ser adotado por reflexo. Pode dificultar rotação de certificados, inspeção legítima e funcionamento em algumas redes. Só deve ser considerado após threat modeling específico e com estratégia operacional de atualização.

7.2. Política de retry

Não fazer retry indiscriminado. Um retry pode gerar custos duplicados ou duas respostas para o mesmo prompt.

Situação	Retry automático	Comportamento
Falha de DNS/conexão antes de enviar body	Sim, limitado	Backoff com jitter
Timeout sem confirmação de processamento	Não por defeito	Mostrar ação manual; avaliar idempotência
HTTP 408	Condicional	Só se a operação for repetível
HTTP 429	Condicional	Respeitar `Retry-After` e limite local
HTTP 5xx	Limitado	Poucas tentativas, sem duplicar cegamente
HTTP 400	Não	Corrigir pedido ou modelo
HTTP 401/403	Não	Rever key e permissões
Quota/billing	Não	Explicar o limite ao utilizador
Cancelamento local	Não	Estado `cancelled`

7.3. Erros normalizados

```
type NormalizedProviderError = {
  code:
    | 'authentication'
    | 'quota'
    | 'rate_limit'
    | 'invalid_request'
    | 'network'
    | 'provider'
    | 'unknown';
  message: string;
  retryable: boolean;
  providerRequestId?: string;
};
```

O `providerRequestId`, quando disponível, pode ajudar o suporte, mas nunca deve incluir uma key, body ou prompt. A UI deve dar uma mensagem acionável e manter os detalhes técnicos fora do texto principal.

8. Interface e experiência de utilização

8.1. Ecrãs do MVP

```
Onboarding
 ├── Proposta de valor e limites de privacidade
 ├── Escolher provider
 ├── Introduzir API key
 ├── Testar configuração
 └── Continuar sem configurar, se o produto o permitir

Home
 ├── Projetos
 ├── Conversas recentes
 ├── Nova conversa
 ├── Pesquisa local opcional
 └── Definições

Projeto
 ├── Nome, descrição e cor
 ├── Conversas agregadas
 ├── Nova conversa neste projeto
 ├── Modelo/provider predefinido opcional
 └── Arquivar ou apagar projeto

Chat
 ├── Mensagens
 ├── Streaming
 ├── Campo de texto
 ├── Microfone para ditar a prompt
 ├── Seletor de provider e modelo no composer
 ├── Modo de resposta no topo: Texto ou Voz
 ├── Cancelar geração
 └── Ações de resposta

Definições
 ├── Providers e API keys
 ├── Voz da IA, idioma, velocidade e TTS
 ├── Privacidade e fluxo de dados
 ├── Custos e preços
 ├── Exportar dados
 └── Apagar todos os dados
```

8.2. Estados essenciais da UI

A UI deve representar explicitamente:

- sem provider configurado;
- key guardada mas não validada;
- validação em curso;
- provider indisponível;
- modelo selecionado e lista de modelos disponível;
- envio em curso;
- resposta em streaming;
- modo de resposta Texto ou Voz selecionado;
- geração cancelada;
- erro recuperável;
- erro de autenticação;
- quota excedida;
- app em background;
- ausência de rede;
- preço desconhecido;
- histórico vazio;
- projeto vazio ou conversa sem projeto;
- eliminação em curso.

8.3. Ações da resposta

O MVP deve suportar, com confirmação quando destrutivo:

- copiar;
- partilhar;
- ouvir/parar TTS;
- alternar o modo de resposta entre Texto e Voz;
- adicionar a conversa a um projeto ou movê-la para outro projeto;
- regenerar;
- apagar;
- editar e reenviar a mensagem do utilizador.

Copiar e partilhar são ações deliberadas de exportação. A política de privacidade deve explicar que, a partir desse momento, o conteúdo pode ser tratado por aplicações ou sistemas fora do controlo da app.

8.4. Adicionar uma conversa a um projeto

O fluxo deve estar disponível em dois pontos:

1. Menu `...` de cada conversa na Inbox ou na lista de um projeto.
2. Menu contextual no topo do chat.

Ao escolher **Adicionar ao projeto** ou **Mover para projeto**, a app abre um bottom sheet com:

- projetos existentes, ordenados por atividade recente;
- nome e número de conversas de cada projeto;
- projeto atual assinalado, quando aplicável;
- ação **Criar novo projeto**;
- opção **Remover do projeto** quando a conversa já estiver agrupada.

Depois da seleção, a app atualiza `project_id`, mantém o histórico e mostra uma confirmação não bloqueante. A conversa não deve ser duplicada nem desaparecer da lista enquanto a operação é concluída. Se a operação falhar, o estado anterior deve ser preservado e a UI deve oferecer nova tentativa.

8.5. Contexto cumulativo de um projeto

Esta é uma regra funcional obrigatória, não uma otimização opcional:

```
Projeto: Lançamento BYOK
  Conversa A: mensagens completas
  Conversa B: mensagens completas
  Nova conversa C: mensagem nova

Pedido enviado ao provider:
  [todas as mensagens de A]
  + [todas as mensagens de B]
  + [mensagem nova de C]
```

Regras de composição:

- incluir todas as conversas que continuam associadas ao projeto e não foram eliminadas;
- incluir as mensagens completas, pela ordem temporal definida e com separadores que identifiquem a conversa de origem;
- incluir instruções ou descrição do projeto antes do histórico, quando existirem;
- incluir o conteúdo novo da conversa no fim do contexto;
- não incluir conversas de outros projetos nem conversas sem projeto;
- não duplicar a nova mensagem no histórico da própria conversa;
- manter o contexto anterior como leitura: responder numa nova conversa não altera as mensagens das conversas de origem;
- mostrar no chat que a resposta está a usar o contexto do projeto e quantas conversas foram incluídas;
- refletir os tokens do contexto cumulativo na estimativa de custo.

O pedido deve ser determinístico e auditável localmente. Se todo o conteúdo não couber na janela do modelo selecionado, a app não pode truncar ou resumir silenciosamente: deve mostrar o tamanho estimado, o limite do modelo e uma ação explícita para o utilizador escolher outro modelo, remover conversas do projeto ou autorizar uma estratégia de compactação documentada. O histórico do projeto permanece completo mesmo quando um pedido individual não pode ser enviado.

Estratégia recomendada: contexto local com handoff OpenAI

Para o OpenAI inicial, a estratégia deve ter duas camadas:

1. **Contexto completo:** a app monta localmente todas as mensagens do projeto e envia-as no `input` da Responses API, juntamente com a nova mensagem, quando o orçamento de tokens permitir.
2. **Handoff/compaction:** quando o contexto completo não couber, a app apresenta a opção explícita de compactar. O endpoint stateless `/responses/compact`, com `store: false`, recebe o contexto completo e devolve uma janela compactada que pode ser usada no pedido seguinte.

O handoff deve ser guardado localmente com:

- hash do conjunto de mensagens que lhe deu origem;
- IDs e contagem das conversas incluídas;
- timestamp e modelo usados na compactação;
- item de compaction opaco devolvido pela OpenAI;
- aviso de que a compactação é uma representação do contexto, não uma cópia legível de todas as mensagens.

Quando uma conversa do projeto for criada, editada, movida ou eliminada, o hash fica inválido e o handoff deve ser reconstruído. A aplicação nunca deve reutilizar um handoff antigo como se ainda representasse todas as conversas atuais.

`previous_response_id` é adequado para continuar uma única conversa linear, mas não deve ser usado como representação de um projeto com várias conversas independentes. A Conversations API também não deve ser adotada por defeito, porque persiste o estado no provider e contraria o princípio de histórico local até existir uma decisão própria sobre retenção.

No beta, a UI deve distinguir claramente:

```
Contexto completo · 3 conversas incluídas
Contexto compactado · baseado em 3 conversas · ver detalhes
```

O segundo estado é aceitável apenas depois de consentimento ou de uma preferência explícita do utilizador. A compactação reduz tokens de entrada e latência, mas pode perder pormenor; por isso, o histórico original mantém-se local e recuperável.

8.6. Performance

Para conversas longas:

- usar `FlatList` bem configurada ou `FlashList` validada para a versão do React Native;
- evitar recalcular todo o Markdown a cada token;
- agrupar atualizações de streaming em intervalos controlados;
- manter um buffer de texto no serviço e publicar deltas para a UI;
- evitar estado duplicado entre store e SQLite;
- paginar o histórico local quando necessário;
- testar memória com uma conversa de pelo menos 1.000 mensagens.

Markdown deve ser renderizado sem HTML executável, sem links capazes de iniciar ações silenciosas e com limites de tamanho. Links externos, se suportados, devem exigir ação explícita e passar por validação de esquema.

9. Voz e acessibilidade

9.1. STT

Fluxo recomendado:

```
Ação explícita no microfone
→ pedir permissão
→ iniciar reconhecimento
→ mostrar transcrição parcial
→ parar
→ colocar texto no editor
→ utilizador rever e editar
→ enviar manualmente
```

A implementação deve documentar se o áudio é processado localmente ou por um serviço do sistema. “Local” não deve ser assumido apenas porque a API é nativa. Devem ser avaliados:

- permissões de microfone;
- permissões de reconhecimento de voz;
- aviso de uso de rede da plataforma;
- comportamento em background;
- idiomas e pontuação;
- transcrição parcial;
- cancelamento;
- dispositivos sem suporte;
- fallback para teclado.

A transcrição nunca deve ser enviada automaticamente. O utilizador deve poder corrigir, apagar, acrescentar contexto ou cancelar.

9.2. TTS

No MVP:

- início manual;
- uma resposta de cada vez;
- botão sempre disponível para parar;
- autoplay desligado por defeito;
- preferência local;
- seleção de idioma e voz quando disponível;
- seleção explícita da voz da IA nas Definições, com nome, idioma e velocidade visíveis;
- início após resposta concluída, não token a token.

O topo do chat deve disponibilizar um controlo segmentado **Texto | Voz**. Em modo Texto, a resposta aparece normalmente na conversa e pode ser ouvida manualmente. Em modo Voz, a resposta deve ser lida pela voz escolhida quando terminar, mantendo sempre uma representação textual acessível e um controlo para parar. O modo selecionado é uma preferência local e não deve alterar silenciosamente o provider ou o modelo.

Se o motor de voz utilizar um serviço externo, esse fluxo deve aparecer na política de privacidade e nas definições.

9.3. Opção gratuita para o beta em português europeu

Para validar voz sem custo por pedido, a recomendação inicial é:

- **TTS:** `expo-speech`, usando as vozes instaladas no iOS ou Android e filtrando `Voice.language` por `pt-PT`;
- **STT:** `expo-speech-recognition`, usando as APIs nativas de reconhecimento e `lang: 'pt-PT'`;
- **fallback:** teclado quando `pt-PT` não estiver disponível ou quando a permissão for recusada.

Esta opção é gratuita para o produto no sentido de não exigir uma API de voz paga. Não garante processamento local: no iOS e em alguns Android, o sistema pode usar rede. O beta deve mostrar essa distinção e testar `requiresOnDeviceRecognition` apenas quando o dispositivo reportar suporte.

Validação mínima:

- `Speech.getAvailableVoicesAsync()` deve devolver uma voz adequada a `pt-PT` para TTS;
- o reconhecimento deve devolver resultados parciais e finais em `pt-PT`;
- Android 13+ deve ser testado com modelo offline instalado e sem esse modelo;
- Android 10–12 deve ser tratado como reconhecimento dependente do serviço disponível, sem prometer offline;
- iOS deve ser testado com Siri/Dictation e permissões de microfone/reconhecimento;
- nenhum áudio deve ser persistido, salvo ação explícita do utilizador;
- a transcrição deve poder ser editada antes de entrar no contexto do projeto.

Se o beta exigir qualidade superior ou comportamento uniforme, a alternativa futura é um provider de STT/TTS pago, mas isso deixa de ser uma validação gratuita e altera o fluxo de privacidade.

9.4. Acessibilidade

Implementar e testar:

- labels acessíveis em todos os botões;
- estados de carregamento anunciáveis sem repetição excessiva;
- contraste adequado;
- suporte ao tamanho de texto do sistema;
- áreas de toque de aproximadamente 44 x 44 pontos;
- VoiceOver;
- TalkBack;
- foco coerente ao abrir teclado, erro ou modal;
- alternativa textual para ações de voz;
- redução de movimento quando aplicável.

9.5. Definições de voz

As Definições devem disponibilizar um seletor de voz que reflita apenas as vozes realmente expostas pelo sistema operativo ou pelo motor escolhido. A app não deve apresentar nomes de vozes que não estejam disponíveis no dispositivo.

Configurações mínimas:

- voz selecionada;
- idioma;
- velocidade;
- botão de pré-escuta;
- iniciar/parar leitura;
- autoplay desligado por defeito;
- fallback para a voz predefinida do sistema;
- aviso quando o idioma da resposta não coincide com a voz escolhida.

O nome “voz da IA” descreve a voz que lê a resposta; não implica que a voz seja gerada pelo provider da mensagem. Se o TTS for fornecido pelo sistema operativo, essa distinção deve aparecer na UI e na política de privacidade.

10. Segurança e threat model

10.1. O que a arquitetura protege

A Opção A reduz a exposição perante:

- um backend próprio que não precisa de receber conversas;
- operadores internos que não têm acesso ao histórico local;
- recolha de conteúdo por analytics, se esta for proibida por implementação;
- bases de dados centrais com prompts e respostas;
- comprometimento de uma API central de proxy que concentrasse keys.

10.2. O que a arquitetura não protege totalmente

Não elimina:

- extração de key de um dispositivo comprometido;
- captura por malware, instrumentation ou debugging privilegiado;
- exposição voluntária através de copiar/partilhar;
- tratamento do prompt pelo provider escolhido;
- retenção definida nos termos do provider;
- dados enviados pelo motor STT, caso este seja remoto;
- backups e screenshots do sistema operativo;
- indisponibilidade, alteração de preços ou mudança de API do provider;
- abuso feito com a própria key do utilizador.

10.3. Matriz de risco

Risco	Probabilidade	Impacto	Mitigação	Risco residual
Key em logs	Média	Alto	Redaction, testes e proibição de body logging	Baixo/Médio
Key extraída de device comprometido	Média	Alto	Secure Store, aviso e curta retenção em memória	Médio/Alto
Prompt em analytics	Média	Alto	Telemetria sem payload e revisão de SDKs	Baixo
Markdown ou link malicioso	Média	Médio	Renderer seguro e esquemas permitidos	Baixo
Pedido duplicado	Média	Médio/Alto	Estados, abort, retry restrito e confirmação	Médio
Quota ou custos inesperados	Média	Alto	Usage, limites locais, aviso e sem retry cego	Médio
STT remoto não documentado	Média	Alto	Inventário de fluxo, consentimento e documentação	Baixo/Médio
Ficheiro SQLite extraído	Baixa/Média	Médio/Alto	Avaliar cifragem e proteção do dispositivo	Médio
Dependência de provider	Alta	Médio	Adapters, catálogo versionado e erros claros	Médio
Backend auxiliar indisponível	Média	Baixo	Fallback local e cache	Baixo
SDK vulnerável	Média	Alto	Auditoria, Dependabot e atualização controlada	Baixo/Médio

10.4. Logs permitidos e proibidos

Permitido:

- versão da app;
- plataforma e versão do sistema;
- provider e modelo por identificador não secreto;
- duração da operação;
- estado genérico;
- código de erro normalizado;
- tamanho aproximado, se não permitir reconstruir conteúdo.

Proibido:

- API keys;
- headers de autorização;
- prompts;
- respostas;
- transcrições;
- áudio;
- request bodies;
- URLs com segredos ou dados pessoais;
- dumps completos de exceções de bibliotecas que incluam requests.

11. Custos e utilização

A app pode apresentar:

- tokens de entrada;
- tokens de saída;
- custo estimado da mensagem;
- custo estimado da conversa;
- número de pedidos;
- data da tabela de preços aplicada.

O texto obrigatório deve ser próximo de:

> Estimativa calculada com base nos dados de utilização devolvidos pelo provider e na tabela de preços disponível na aplicação. Não representa necessariamente a faturação oficial.

11.1. Regras de cálculo

- usar usage real quando devolvido;
- distinguir input e output;
- suportar preços por modelo e por modalidade;
- suportar cached input, batch ou outras categorias quando aplicável;
- guardar a versão/data da tabela usada no cálculo;
- usar fallback local quando o backend estiver indisponível;
- apresentar “desconhecido” quando não houver dados suficientes;
- não bloquear o chat por falta de preço;
- não afirmar precisão quando o provider arredondar ou calcular de forma diferente.

Fórmula simplificada para modelos com duas tarifas:

$$ Custo = \frac{tokens_{entrada}}{1,000,000} \times preco_{entrada} + \frac{tokens_{saida}}{1,000,000} \times preco_{saida} $$

A fórmula não deve ser aplicada a modelos cuja estrutura de preços não seja compatível sem uma regra específica.

11.2. Controlo local

Como a app usa a key do utilizador, pode não conseguir impor um limite financeiro real no provider. Pode, no entanto, oferecer:

- limite local de tokens por pedido;
- confirmação para prompts longos;
- aviso antes de regenerar;
- orçamento estimado local;
- contagem por conversa;
- link para a página oficial de billing do provider.

Esses controlos são auxiliares e não substituem os limites configurados na conta do provider.

12. Compliance, privacidade e distribuição

Antes da publicação, devem ser preparados:

- política de privacidade em português e inglês;
- termos de utilização;
- contacto de suporte;
- informação sobre BYOK e faturação direta;
- declaração de fluxo para cada provider;
- declaração do fluxo de STT e TTS;
- processo para pedidos de eliminação e suporte;
- inventário de SDKs e subprocessadores;
- classificação de dados e prazos de retenção;
- revisão de requisitos das App Store e Google Play;
- verificação de termos de utilização dos providers.

A documentação deve distinguir claramente:

- dados tratados pela app;
- dados enviados diretamente ao provider;
- dados eventualmente tratados pelo sistema operativo ou motor de voz;
- metadados técnicos do backend auxiliar;
- dados exportados pelo próprio utilizador.

Evitar afirmações como:

```
 100% impossível de hackear
Segurança total
Nenhum dado sai do telemóvel
```

A frase “nenhum dado sai do telemóvel” é incorreta quando o utilizador envia texto ao provider ou quando STT/TTS usa serviços externos.

13. ASO e website

13.1. ASO

Preparar:

- nome curto e memorável;
- subtítulo;
- descrição curta e longa;
- palavras-chave em português e inglês;
- screenshots com situações reais da app;
- vídeo de demonstração;
- ícone reconhecível;
- categoria adequada;
- explicação visível de BYOK;
- explicação transparente de que o utilizador paga diretamente os providers;
- respostas a reviews e canal de suporte.

Posicionamentos possíveis:

```
 Chat privado com as suas próprias API keys
Cliente de IA com voz para iPhone e Android
Converse diretamente com os seus providers de IA
```

13.2. Website público

O website deve existir antes ou juntamente com o lançamento público e incluir:

- página do produto;
- documentação de configuração;
- política de privacidade;
- perguntas frequentes;
- providers suportados;
- tutorial para cada provider;
- página de segurança com limitações honestas;
- custos e responsabilidade de faturação;
- contacto;
- changelog;
- versões em português e inglês.

Boas práticas:

- URLs legíveis;
- títulos e meta descriptions únicos;
- Schema.org `SoftwareApplication` quando adequado;
- sitemap.xml;
- robots.txt;
- canonical URLs;
- Open Graph;
- acessibilidade;
- performance elevada;
- conteúdo factual e atualizado.

14. Testes e qualidade

14.1. Testes unitários

Cobrir:

- adapters e serialização de requests;
- parsing de SSE fragmentado;
- normalização de erros;
- cálculo de custo;
- validação de configuração;
- migrações SQLite;
- criação e renomeação de conversas;
- transições de estado;
- cancelamento;
- sanitização de logs;
- rejeição de secrets em URLs.

14.2. Testes de integração

Testar:

- Secure Store em iOS e Android;
- SQLite e eliminação em cascata;
- permissões de microfone;
- STT parcial, cancelamento e fallback;
- TTS play/stop;
- requests reais num ambiente controlado;
- erros 400, 401, 403, 408, 429 e 5xx;
- perda e recuperação de rede;
- entrada em background;
- rotação e tamanhos de ecrã;
- fallback quando o backend auxiliar está indisponível.

14.3. Testes E2E

Fluxos mínimos:

1. instalação nova;
2. onboarding;
3. configuração de key;
4. validação da key;
5. criação de conversa;
6. envio;
7. streaming;
8. cancelamento;
9. regeneração;
10. edição e reenvio;
11. ditado e edição da transcrição;
12. TTS e interrupção;
13. apagar conversa;
14. apagar todos os dados;
15. restart e recuperação do histórico.

14.4. Matriz de dispositivos

Testar, pelo menos:

- iPhone físico recente;
- iPhone com iOS 15;
- Android recente;
- Android 10;
- dispositivo de baixo desempenho;
- VoiceOver;
- TalkBack;
- idiomas de voz relevantes;
- Wi-Fi;
- rede móvel;
- rede instável;
- modo de texto grande;
- modo de bateria reduzida, quando afete background ou voz.

14.5. Critérios de qualidade

O MVP não deve avançar para distribuição pública se:

- existir uma key em logs, bundle, SQLite ou estado persistido;
- uma mensagem puder chegar ao backend próprio;
- cancelar uma geração deixar uma mensagem a aguardar indefinidamente;
- uma resposta parcial for apresentada como concluída sem confirmação;
- erros de auth e quota forem indistinguíveis;
- o TTS não puder ser parado;
- a transcrição for enviada sem revisão;
- o histórico desaparecer após restart;
- VoiceOver ou TalkBack não conseguirem operar o fluxo principal;
- a política de privacidade não refletir o comportamento real.

15. CI/CD e ambientes

15.1. Ambientes

```
local
development
staging
production
```

Cada ambiente deve definir:

- catálogo e preços;
- endpoints auxiliares;
- feature flags;
- logging;
- bundle identifiers;
- credenciais de assinatura;
- política de crash reporting;
- versão mínima suportada.

Nenhuma API key real de utilizador deve ser embutida em builds, fixtures ou configurações de CI.

15.2. Pipeline

```
Pull Request
 ├── typecheck
 ├── ESLint
 ├── Prettier check
 ├── testes unitários
 ├── auditoria de dependências
 ├── validação de secrets
 └── validação de build

Merge
 ├── preview/development build
 └── QA interno

Release
 ├── production build assinado
 ├── TestFlight
 ├── Google Play internal testing
 └── checklist de privacidade e versão
```

Expo Go pode ser útil para UI e navegação, mas não deve ser a única validação para Secure Store, SQLite, permissões, STT ou qualquer módulo nativo. O desenvolvimento deve usar EAS development builds desde a fase de validação técnica.

16. Plano de execução

Fase 0 — Validação técnica

Estimativa: 3–5 dias úteis

- confirmar versões suportadas de Node, Expo e Xcode/Android tooling;
- criar development builds;
- validar Secure Store;
- validar SQLite e migrações;
- testar STT candidato em dispositivos físicos;
- testar `expo-speech`;
- executar uma chamada direta ao provider;
- confirmar streaming, cancelamento e background;
- documentar o fluxo real de dados.

Gate: ditar, rever, enviar em streaming, cancelar e ouvir a resposta em iOS e Android.

Fase 1 — Fundação

Estimativa: 1–2 semanas

- navegação;
- tema e componentes base;
- persistência local;
- Secure Store;
- Zustand sem persistência de secrets;
- onboarding;
- registry e primeiro adapter;
- tratamento global de erros;
- logging redigido.

Fase 2 — Chat

Estimativa: 2–3 semanas

- lista de mensagens;
- streaming;
- cancelamento;
- estados persistidos;
- Markdown seguro;
- regeneração;
- copiar e partilhar;
- histórico;
- renomear e apagar;
- custos estimados.

Fase 3 — Voz

Estimativa: 1–2 semanas

- permissões;
- STT;
- transcrição editável;
- TTS manual;
- play/stop;
- seleção de vozes;
- acessibilidade;
- testes físicos.

Fase 4 — Qualidade e lançamento

Estimativa: 2–3 semanas

- E2E;
- performance;
- segurança;
- revisão de dependências;
- textos legais;
- ASO;
- builds de produção;
- TestFlight;
- Google Play testing;
- correção de bugs.

16.1. Estimativa global

Escopo	Equipa	Estimativa
Validação técnica	1 developer	3–5 dias
Protótipo técnico	1 developer	1–2 semanas
MVP com um provider	1 developer experiente	6–10 semanas
MVP com dois ou três providers	1 developer experiente	8–14 semanas
MVP com pequena equipa	2 developers	6–10 semanas
Backend auxiliar	adicional	1–2 semanas
Contas e sincronização	adicional	3–6 semanas

O maior risco de estimativa está na biblioteca de STT, na compatibilidade do streaming, nas diferenças entre APIs de providers e nos testes reais em dispositivos.

17. Decisões finais recomendadas

Tema	Decisão
Arquitetura	Opção A, chamadas diretas aos providers
Backend	Opcional, apenas catálogo, preços, configuração e health check
Provider inicial	OpenAI Responses API com `gpt-5.6-luna`
Keys	Secure Store, nunca em SQLite ou estado persistido
Histórico	SQLite local
Estado	Zustand apenas para runtime e preferências não sensíveis
STT	`expo-speech-recognition`, `pt-PT`, com transcrição editável e envio explícito
TTS	`expo-speech`, voz do sistema em `pt-PT`, play/stop e autoplay desligado
Contexto de projetos	Histórico local completo; input completo quando couber; handoff/compaction stateless quando necessário
Cifragem do histórico	Proteção do dispositivo no beta; reavaliar cifragem adicional depois do beta
Streaming	Obrigatório no MVP, normalizado por eventos internos
Retries	Limitados e condicionais, nunca em auth/quota
Custos	Estimativas com usage e tabela versionada
Conta	Não necessária no MVP
Build	EAS development build desde a validação técnica
Privacidade	Sem conteúdo no backend próprio ou analytics
Compliance	Política de privacidade alinhada com providers, sistema operativo e voz

18. Questões em aberto antes do desenvolvimento

Estas questões devem ter uma decisão registada:

1. Qual versão da Responses API e que capacidades de `gpt-5.6-luna` serão fixadas no beta?
2. O produto permite entrar sem key ou exige configuração no onboarding?
3. `expo-speech-recognition` suporta efetivamente `pt-PT` nos dispositivos-alvo?
4. O STT ficará em modo on-device quando suportado ou aceitará reconhecimento por rede?
5. O backend auxiliar terá analytics técnicos? Que metadados serão recolhidos?
6. Qual é a política para backups e exportação?
7. Que modelos e preços estarão no fallback local?
8. Qual o limite local de tamanho de prompt e histórico?
9. O produto permitirá links Markdown e partilha para outras apps?
10. Quem fará suporte quando a key, quota ou billing falhar no provider?
11. Quais idiomas serão suportados no lançamento?
12. Quais dispositivos físicos estarão disponíveis para QA?
13. Que requisitos legais e de lojas se aplicam à introdução e tratamento de API keys?
14. Depois do beta, o histórico local precisa de cifragem adicional além da proteção do dispositivo?

19. Critérios de aceitação do MVP

O MVP será tecnicamente aceitável quando:

- nenhuma API key for armazenada em texto simples fora do Secure Store;
- nenhuma key surgir em logs, crash reports, bundle, URL ou payload de analytics;
- nenhuma mensagem, resposta, transcrição ou áudio for enviado para o backend próprio;
- o chat funcionar diretamente com pelo menos um provider;
- o streaming puder ser cancelado e persistido corretamente;
- erros de autenticação, quota, rate limit, rede e modelo inválido forem distinguíveis;
- conversas persistirem após reiniciar a app;
- o utilizador puder apagar todo o histórico e keys locais;
- uma conversa puder ser adicionada ou movida para um projeto a partir da Inbox e do topo do chat;
- uma nova conversa criada dentro de um projeto enviar todas as conversas desse projeto, além da mensagem nova, como contexto explícito;
- o chat mostrar o projeto e o número de conversas usadas no contexto;
- a app impedir o envio quando o contexto completo exceder o limite sem apresentar uma decisão explícita ao utilizador;
- a transcrição puder ser editada antes do envio;
- o composer tiver um microfone visível para ditar a prompt;
- o TTS puder ser iniciado e interrompido;
- o utilizador puder escolher e pré-escutar a voz da IA nas Definições;
- VoiceOver e TalkBack conseguirem executar o fluxo principal;
- a app funcionar em iOS e Android suportados;
- houver fallback local para configuração essencial;
- a app funcionar sem conta;
- custos desconhecidos forem apresentados como desconhecidos, não como zero;
- a documentação de privacidade refletir exatamente o comportamento real.

20. Conclusão

A Opção A é a melhor escolha quando o valor central é BYOK genuíno e privacidade perante servidores próprios. É uma arquitetura simples de explicar, mas não é uma arquitetura sem riscos: a key é usada no dispositivo, o provider recebe diretamente o conteúdo, e a aplicação não controla integralmente o sistema operativo, o runtime ou as políticas de terceiros.

A recomendação final é avançar primeiro com uma **validação técnica curta e mensurável**, seguida de um MVP com um provider, streaming, histórico local, Secure Store e voz manual. O backend auxiliar deve permanecer opcional e fora do caminho crítico.

O produto poderá ser comunicado como uma aplicação que reduz a exposição perante a infraestrutura do operador. Não deve ser comunicado como uma garantia de anonimato, segurança absoluta, processamento local de toda a voz ou ausência total de transferência de dados.

21. Revisão crítica

Esta secção é uma revisão adversarial da proposta. O objetivo é identificar afirmações que poderiam falhar em produção, decisões que ainda não estão fechadas e riscos que a primeira versão poderia esconder.

21.1. Achados prioritários

Severidade	Achado	Consequência	Correção recomendada
Crítica	Uma key no dispositivo permite abuso e faturação fora do controlo do operador	O utilizador pode sofrer custos inesperados se o dispositivo ou a key forem comprometidos	Explicar o risco no onboarding, recomendar limites no provider, oferecer limites locais e nunca prometer proteção absoluta
Alta	O STT não define se o áudio é local ou enviado para um serviço	A promessa de privacidade pode ser falsa e o consentimento pode ser insuficiente	Escolher a biblioteca depois de mapear o fluxo de áudio por plataforma e atualizar a política de privacidade
Alta	O contrato `AIProviderAdapter` referencia `RequestOptions`, `ValidationResult` e `ChatResponse` sem os definir	Diferentes adapters podem interpretar o contrato de forma incompatível	Publicar tipos completos, capacidades do modelo e uma política de erro versionada antes do primeiro adapter
Média / pós-beta	SQLite local não significa cifragem em repouso	Uma cópia do sandbox pode expor o histórico	No beta, depender da proteção do dispositivo e documentar o risco; avaliar SQLCipher ou alternativa compatível depois
Alta	"Sem dados no backend" não é verificável apenas por intenção	Um SDK, endpoint de configuração ou crash reporter pode recolher conteúdo	Implementar allowlist de rede, testes de proxy, revisão de SDKs e logs sem payload
Alta	Preços e catálogo remotos podem ficar desatualizados ou ser adulterados	Custos estimados incorretos e configuração insegura	Assinar ou validar configuração, versionar tabelas, manter fallback e mostrar sempre a data da estimativa
Média	O beta junta contexto cumulativo de projetos, voz, streaming, custos, partilha e vários fluxos de histórico	A estimativa de 6–10 semanas fica otimista e aumenta o risco de lançamento	Fixar um provider, limitar o beta a um modelo e validar o handoff antes de adicionar providers ou plugins
Média	A ausência de conta implica perda ao trocar de dispositivo ou desinstalar	Pode gerar perceção de perda de dados ou suporte elevado	Explicar o modelo local, disponibilizar exportação/importação cifrada e confirmar antes de apagar
Média	Não existe direção visual nem especificação de estados	A implementação pode divergir entre ecrãs e parecer um cliente genérico	Usar o apêndice visual deste documento como referência de composição, tokens e estados

21.2. Inconsistências corrigidas

Contrato incompleto

O contrato anterior parecia concreto, mas não era implementável sem inventar tipos. A versão de implementação deve definir pelo menos:

```
 type RequestOptions = {
  apiKey: () => Promise<string>;
  signal?: AbortSignal;
  timeoutMs?: number;
};

type ValidationResult = {
  valid: boolean;
  code?: 'authentication' | 'quota' | 'network' | 'provider' | 'unknown';
  message?: string;
};

type ChatResponse = {
  message: ChatMessage;
  usage?: Usage;
  finishReason?: 'stop' | 'length' | 'cancelled' | 'unknown';
};
```

O exemplo é um ponto de partida, não um contrato final. Em particular, `apiKey` deve ser uma função de acesso de curta duração e não o valor da key armazenado no estado da interface. O contrato também deve prever capacidades: visão, ferramentas, tamanho máximo, modalidades e suporte a streaming.

Privacidade do áudio

"STT nativo" não significa necessariamente "STT local". A API da plataforma pode enviar áudio ou transcrição para um serviço operado pelo sistema. Até a biblioteca e o comportamento por idioma estarem validados, a comunicação correta é "entrada de voz sujeita às políticas da plataforma e do motor de reconhecimento".

Backend e metadados

O backend pode não receber prompts e ainda assim observar IP, user-agent, timestamps e padrões de acesso. A frase "sem conteúdo no backend" é defensável; "sem recolha de dados" não é. A política deve separar conteúdo, credenciais e metadados técnicos.

Streaming e retry

Um timeout não informa se o provider terminou a geração. Por isso, retry automático após o body ser enviado pode duplicar custos. A implementação deve distinguir "não foi enviado", "foi enviado mas não confirmado" e "foi concluído". Se não houver idempotência suportada pelo provider, a ação manual é mais segura.

21.3. Reavaliação do âmbito

Para reduzir risco, o lançamento deve ser dividido:

Entrega	Incluído	Condição de saída
MVP-0 técnico	Um provider, key, request, streaming, cancelamento, Secure Store	Funciona em dois dispositivos físicos
MVP-1 utilizável	Histórico local, erros, regeneração, custos estimados, TTS manual	Passa critérios de aceitação e acessibilidade
MVP-2 voz	STT editável, idiomas definidos, consentimento e fallback	Fluxo de áudio documentado por plataforma
Pós-MVP	Outros providers, anexos, sync, agentes e hands-free	Requisitos, threat model e custos próprios

Esta divisão mantém a hipótese principal testável: uma pessoa consegue conversar com o seu provider usando a sua própria key, com controlo local sobre o histórico.

22. Direção de design do produto

22.1. Princípio visual

A referência é a clareza e calma de um cliente como o Claude: muito espaço, uma conversa como foco principal, navegação discreta e ações que aparecem no contexto. A execução proposta não deve copiar marca, componentes, textos ou composição proprietários. A identidade do BYOK Chat será construída em torno da ideia de **controlo sereno**:

- superfície de argila clara, em vez de branco clínico;
- terracota profundo para leitura prolongada e navegação;
- terracota vivo para ações e estados de atenção;
- pêssego queimado para confirmação e superfícies secundárias;
- tipografia editorial para títulos e sans-serif funcional para controlos;
- composição aberta, sem cartões empilhados dentro de cartões;
- uma ação primária por vista;
- informação de provider visível, sem dominar a conversa.

22.2. Tokens visuais

Token	Valor	Uso
`canvas`	`#FCF5F0`	Fundo geral, argila muito clara
`surface`	`#FFFCF9`	Campo de composição e superfícies elevadas
`ink`	`#4A302A`	Texto principal, terracota suave
`muted`	`#96776B`	Texto secundário
`terracotta-deep`	`#945445`	Navegação, foco e títulos
`terracotta`	`#D98168`	Ação primária e estados de atenção
`clay`	`#E4A38C`	Ícones e estados secundários
`peach`	`#F1C2B0`	Confirmação, seleção e superfícies suaves
`line`	`#ECDAD2`	Divisores e bordas
`danger`	`#AA4D45`	Erros destrutivos

Tipografia recomendada para o design: **Newsreader** para títulos de ecrã e **DM Sans** para interface, conteúdo e números. A implementação mobile deve incluir os ficheiros de fonte no bundle ou escolher equivalentes com métricas verificadas; não deve depender de uma fonte remota para o primeiro arranque.

22.3. Paleta selecionável pelo utilizador

As Definições devem incluir uma secção **Aparência** com uma paleta curta de variações terracota. A escolha altera acentos, botões, seleção e ilustrações, mas preserva contraste e significado semântico. Não deve permitir combinações arbitrárias que prejudiquem leitura ou acessibilidade.

Paletas iniciais:

Nome	Fundo	Acento	Texto
Terra	`#FCF5F0`	`#D98168`	`#4A302A`
Barro	`#F8ECE6`	`#B96855`	`#4A302A`
Pôr do sol	`#FFF6EF`	`#E4A38C`	`#5F3D34`
Romã	`#FAEDEA`	`#A9554D`	`#4A302A`

O utilizador escolhe uma paleta através de swatches com nome acessível, estado selecionado e pré-visualização imediata. A aplicação deve verificar contraste WCAG para texto, controlos e estados de erro em cada combinação. A paleta é uma preferência local e não é enviada para providers ou para o backend.

22.4. Projetos no design system

Projetos são uma camada de organização visível entre a Inbox e a conversa. Na Inbox, a ordem recomendada é **Projetos** primeiro e **Conversas recentes** depois. Um projeto aparece como uma linha agrupada, não como um cartão decorativo dentro de outro cartão, com:

- ponto de cor da paleta escolhida;
- nome do projeto;
- número de conversas;
- data da última atividade;
- ação “Abrir projeto” e menu contextual;
- menu da conversa com “Adicionar ao projeto” ou “Mover para projeto”;
- indicador “Contexto: todas as conversas” ao iniciar uma conversa no projeto;
- estado vazio com “Adicionar conversa”.

Exemplo de conteúdo visual:

```
PROJETOS                                  Ver todos

  • Lançamento BYOK                 3 conversas · agora
    Produto, segurança e lançamento

CONVERSAS RECENTES
  Plano de lançamento                  agora
```

Dentro do projeto, o cabeçalho mostra nome, descrição curta e provider/modelo predefinido. As conversas continuam a ser objetos independentes: podem ser movidas para outro projeto, retiradas do projeto ou pesquisadas localmente. O projeto não deve esconder o acesso à conversa nem obrigar o utilizador a criar uma estrutura antes de começar.

Ao tocar em **Nova conversa neste projeto**, o composer deve mostrar uma confirmação persistente e discreta, por exemplo: `Contexto do projeto · 3 conversas incluídas`. Esse indicador abre os detalhes do contexto e deixa claro que a resposta usará todas as conversas do projeto, não apenas a última.

22.5. Plugins como plus pós-MVP

Plugins podem ser uma extensão futura, mas não devem fazer parte do MVP nem receber acesso implícito às keys ou ao histórico. Numa arquitetura BYOK direta, um plugin com acesso livre poderia exfiltrar exatamente os dados que a arquitetura tenta proteger.

Uma evolução aceitável deve começar por extensões declarativas e limitadas:

- manifesto de capacidades e permissões;
- consentimento por plugin e por projeto;
- acesso apenas ao contexto selecionado pelo utilizador;
- nenhuma API key exposta ao plugin;
- sandbox ou processo isolado;
- revogação e eliminação de dados;
- indicação clara quando uma ação envia dados para um serviço externo;
- revisão de segurança e compatibilidade com os termos dos providers.

Plugins que executem código arbitrário, ferramentas externas ou chamadas com dados privados exigem um threat model próprio, auditoria, UX de consentimento e provavelmente uma arquitetura de execução separada. Devem ser tratados como pós-MVP, não como simples botão de marketplace.

22.6. Regras de composição

- O conteúdo útil começa abaixo da safe area e nunca fica atrás do teclado.
- O cabeçalho tem uma ação de navegação e uma ação contextual; o resto fica em menu.
- A conversa é uma coluna de leitura, com largura confortável e sem bolhas coloridas por defeito.
- Mensagens do utilizador usam uma superfície ligeiramente contrastante; respostas usam o canvas.
- O composer fica ancorado ao fundo, cresce até um limite e mostra, no mesmo grupo de ações, um microfone minimalista e o botão de enviar.
- O microfone é um ícone de contorno simples, sem círculo decorativo ou texto persistente; fica imediatamente à esquerda do submit, com área de toque independente e label acessível “Ditar prompt”.
- O composer apresenta uma linha compacta para escolher provider e modelo; tocar nessa linha abre um bottom sheet com capacidades e contexto, sem deslocar a mensagem em edição.
- O topo do chat apresenta um controlo segmentado Texto | Voz. O estado ativo é visível e o modo Voz nunca esconde a resposta textual.
- O estado de streaming usa cursor e ação “Parar”; não usa spinner como único indicador.
- Os estados vazios explicam a próxima ação com uma frase e um botão, sem transformar o ecrã numa landing page.
- Modais destrutivos usam texto explícito; ações frequentes usam ícones reconhecíveis e tooltips quando suportados.

22.7. Estados que o design deve cobrir

Vista	Estado normal	Estado vazio	Estado de erro	Estado em curso
Inbox	Conversas recentes e provider ativo	“Comece uma conversa”	Falha ao carregar histórico	Migração ou eliminação
Projetos	Projetos e contagem de conversas	“Crie o seu primeiro projeto”	Falha ao abrir ou atualizar projeto	Arquivar ou mover conversa
Chat	Mensagens e composer	Sugestões de primeiro prompt	Auth, quota, rede ou modelo	Streaming com “Parar”
Provider	Key configurada com estado	Nenhum provider	Key rejeitada ou indisponível	Validação direta
Settings	Preferências e privacidade	Valores por defeito	Falha ao exportar/apagar	Exportação ou limpeza

23. Apêndice visual — ecrãs conceptuais

O PDF inclui quatro ecrãs conceptuais mobile, produzidos para este relatório como uma prancha de direção visual. São referências de layout e interação, não um ficheiro Figma final nem uma especificação de pixels para implementação.

23.1. Ecrã Inbox e Projetos

Objetivo: mostrar que a app está pronta para uma conversa sem obrigar o utilizador a compreender a arquitetura. A primeira secção é **Projetos**, com "Lançamento BYOK" e o número de conversas agregadas; abaixo ficam as conversas recentes sem projeto. O menu `...` de cada conversa permite **Adicionar ao projeto**. Ao criar uma conversa no projeto, o design deve mostrar `Contexto: todas as conversas · 3 incluídas` antes da primeira prompt.

23.2. Ecrã Chat

Objetivo: fazer a conversa ocupar o centro da experiência. O topo permite escolher Texto ou Voz; o composer permite escolher o modelo e ditar com o microfone. O streaming não altera o layout e o modo Voz mantém sempre uma resposta textual acessível.

23.3. Ecrã Configurar provider

Objetivo: tornar a confiança verificável. A key fica mascarada, o armazenamento local é explicado em linguagem curta e a validação é uma ação explícita. O ecrã inclui uma imagem original da identidade BYOK Chat, criada para esta proposta, em vez de depender de ilustração de terceiros.

23.4. Ecrã Definições, voz e aparência

Objetivo: substituir promessas vagas por informação operacional: voz "Sofia", idioma, reprodução de respostas, paleta terracota selecionada, localização das conversas, destino do prompt e data da tabela de preços.

Os quatro ecrãs estão anexados visualmente ao PDF e devem ser usados como referência conjunta com os tokens, estados, imagem original, projetos e regras de acessibilidade desta secção. O mockup da Inbox mostra explicitamente a camada de Projetos.

Referências técnicas consultadas

Consultadas em 2 de setembro de 2026:

- OpenAI — Models: identificação, posicionamento, janela de contexto e preços de `gpt-5.6-luna`.
- OpenAI — Conversation state: `input`, `previous_response_id`, Conversations API e limites de contexto.
- OpenAI — Compaction: compaction server-side, endpoint stateless `/responses/compact` e uso de `store: false`.
- Expo — Speech: vozes disponíveis no dispositivo, locale, `voice`, `rate`, `speak` e `stop`.
- expo-speech-recognition — npm: reconhecimento nativo Expo/iOS/Android, permissões, resultados parciais, locales e suporte on-device dependente do dispositivo.

As referências indicam capacidades da documentação consultada; a disponibilidade efetiva do modelo, da voz `pt-PT` e dos serviços de reconhecimento deve ser verificada no ambiente do beta.

BYOK CHAT / DESIGN DIRECTION

Projeto como contexto.

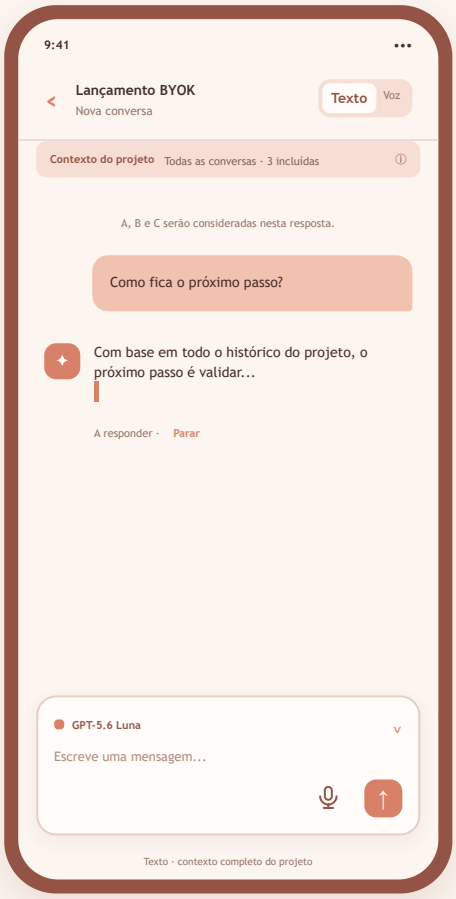
Prancha conceptual mobile
02 setembro 2026

01 / INBOX + PROJETOS



Projetos são visíveis e explicam o contexto que será herdado.

02 / NOVA CONVERSA NO PROJETO



O utilizador vê as 3 conversas incluídas antes de enviar.

03 / PROVIDER



04 / DEFINIÇÕES



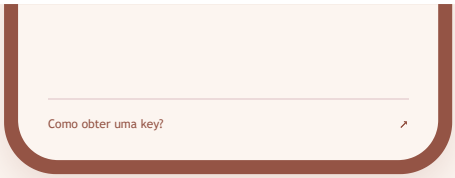
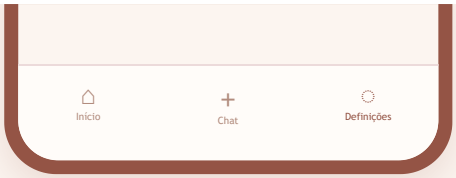


Imagem própria e modelo inicial definido no contexto do beta.



Voz, aparência e contexto permanecem controláveis.